

Лабораторная работа № 3

Реализация клиентской части веб-приложения: фотографии, модальные окна, бургер-меню, состояние пользователя и карта

1. Место работы в курсе

В третьей лабораторной работе студент развивает многостраничное приложение из лабораторной работы № 2 и реализует основные интерактивные элементы клиентской части.

Структура App Router, динамический маршрут и учебный DemoAuthProvider сохраняются. Каталог предметных сущностей дополняется локальными фотографиями и координатами. На странице появляются карточки, модальное окно подробного просмотра, бургер-меню и интерактивная карта с метками.

В TheraCity сходные задачи решают компоненты BurgerMenu, PhotoModals, OptimizedPhoto и PhotoMap. В учебной работе воспроизводится общий архитектурный подход: интерактивность изолируется в Client Components, фотографии выводятся через next/image, модальное окно связано с выбранной сущностью, а карта строится по её координатам. Код и предметная область TheraCity не копируются.

Учебное состояние пользователя из лабораторной работы № 2 применяется для условного отображения действия «Добавить в избранное». Это по-прежнему не серверная авторизация. Избранное и пользователь сбрасываются после обновления страницы, а защищённая операция на сервере ещё отсутствует.

2. Цель работы

Разработать интерактивную клиентскую часть веб-приложения на Next.js и React: вывести фотографии предметных сущностей, реализовать доступное модальное окно, создать бургер-меню, связать интерфейс с учебным состоянием пользователя и отобразить объекты на карте с помощью JavaScript API Яндекс Карт.

3. Задачи работы

В ходе выполнения работы студент должен:

- дополнить тип предметной сущности фотографией и координатами;
- подготовить локальные изображения в каталоге public;
- вывести изображения с помощью next/image;
- создать интерактивную галерею карточек;
- управлять выбранной карточкой с помощью useState;
- реализовать модальное окно через createPortal;
- обеспечить закрытие модального окна мышью и клавишей Escape;
- управлять клавиатурным фокусом внутри модального окна;
- создать бургер-меню и определить активный маршрут;
- показать разные действия для гостя и учебного пользователя;

- подключить JavaScript API Яндекс Карт;
- создать карту и метки по координатам сущностей;
- освобождать внешние ресурсы в `cleanup`-функциях `useEffect`;
- проверить сборку и основные пользовательские сценарии.

4. Теоретическая часть

4.1 Клиентские компоненты и интерактивность

В App Router страницы и layouts по умолчанию являются Server Components. Официальная документация Next.js рекомендует использовать Client Components, когда необходимы состояние, обработчики событий, эффекты, браузерные API или пользовательские хуки. Граница клиентского модуля объявляется директивой "use client" перед импортами. (Next.js: Server and Client Components; Next.js: use client)

```
"use client";

import { useState } from "react";
```

Не вся страница обязана становиться клиентской. Серверный `page.tsx` может импортировать небольшой клиентский компонент `Gallery`. Такой подход сохраняет статическую оболочку серверной и отправляет в браузер JavaScript только для интерактивной части.

```
import PlacesGallery from "../_components/PlacesGallery";

export default function GalleryPage() {
  return <PlacesGallery />;
}
```

4.2 Состояние и условный рендеринг

Официальная справка React определяет `useState` как хук для добавления переменной состояния. `useState` возвращает текущее значение и функцию обновления, после вызова которой React выполняет повторный рендер.

```
const [selected, setSelected] = useState<StudyPlace | null>(null);
```

Когда пользователь нажимает карточку, `selected` получает объект. Когда окно закрывается, `selected` становится `null`.

Официальный раздел React Conditional Rendering указывает, что компоненты могут выбирать JSX обычными средствами JavaScript: `if`, условным оператором `&&`.

```
{selected ? (
  <PhotoModal place={selected} onClose={() => setSelected(null)} />
) : null}
```

Условный рендеринг отвечает только за наличие интерфейса. Например, условие `user ? <button> : <Link>` не является серверной проверкой прав.

4.3 Обработчики событий

Официальная документация React Responding to Events указывает, что обработчик передаётся JSX-элементу как функция. Для действия необходимо использовать семантический элемент `button`, а не `div` с `onClick`: кнопка имеет встроенную поддержку клавиатуры.

```
<button type="button" onClick={() => setSelected(place)}>  
  Открыть карточку  
</button>
```

Обработчики событий React распространяются вверх по дереву. Если щелчок по затемнённом фону закрывает модальное окно, щелчок внутри окна не должен случайно вызвать тот же обработчик. Для разделения действий можно проверить `event.target === event.currentTarget` или остановить распространение в контейнере диалога.

4.4 Эффекты и очистка ресурсов

Согласно официальной справке React `useEffect`, эффект используется для синхронизации компонента с внешней системой. В этой работе внешними системами являются события `window`, стили `document`, DOM-фокус и экземпляр карты.

```
useEffect(() => {  
  function handleKeyDown(event: KeyboardEvent) {  
    if (event.key === "Escape") onClose();  
  }  
  
  window.addEventListener("keydown", handleKeyDown);  
  return () => window.removeEventListener("keydown", handleKeyDown);  
}, [onClose]);
```

Функция, возвращаемая из `useEffect`, является `cleanup`. Она должна отменить подписку, восстановить изменённое состояние документа или уничтожить экземпляр внешней библиотеки.

В режиме разработки React Strict Mode может выполнить дополнительный цикл `setup` → `cleanup` → `setup`. Официальная документация объясняет, что это проверяет полноту очистки. Компонент не должен создавать дублирующие обработчики или карты.

4.5 Ссылки на DOM через `useRef`

Официальная справка React `useRef` определяет `ref` как ссылку на значение, которое не требуется для рендеринга. Изменение `ref.current` не запускает повторный рендер.

В клиентском интерфейсе `ref` используется для:

- ссылки на кнопку закрытия модального окна;
- возврата фокуса на карточку, открывшую окно;
- ссылки на HTML-контейнер карты;
- хранения последнего интерактивного элемента.

```
const closeButtonRef = useRef<HTMLButtonElement>(null);
```

После открытия диалога можно вызвать `closeButtonRef.current?.focus()`. После закрытия фокус возвращается к элементу, который открыл диалог.

4.6 Порталы и модальные окна

Официальная справка React `createPortal` указывает, что портал размещает DOM-узлы в другом месте документа, при этом JSX остаётся дочерним элементом того же React-дерева. Портал удобен для модального окна, которое должно находиться поверх контейнеров с `overflow` или локальным `stacking context`.

```
import { createPortal } from "react-dom";

return createPortal(
  <div className={styles.overlay}>...</div>,
  document.body,
);
```

События из портала распространяются по React-дереву, а контекст остаётся доступным. Поэтому модальное окно может использовать `DemoAuthProvider`, хотя физически его DOM размещён в `body`.

Документация React отдельно предупреждает, что портал должен быть доступным и требует управления фокусом. Для требований диалога используется официальный `WAI-ARIA Authoring Practices Guide W3C`.

4.7 Доступность модального диалога

Официальный `WAI-ARIA Dialog (Modal) Pattern` определяет модальный диалог как окно поверх основного содержимого, при котором взаимодействие с нижележащим содержимым недоступно.

Контейнер модального окна должен иметь:

```
role="dialog"
aria-modal="true"
aria-labelledby="place-modal-title"
```

Основные требования клавиатурного взаимодействия из `WAI-ARIA APG`:

- при открытии фокус перемещается внутрь диалога;
- `Tab` и `Shift + Tab` не выводят фокус за границы диалога;
- `Escape` закрывает диалог;
- после закрытия фокус обычно возвращается к элементу, открывшему диалог;
- в диалоге должна быть видимая кнопка закрытия.

`aria-modal` не заменяет реальную логику фокуса. Наличие атрибута без ограничения `Tab` не делает окно полностью доступным.

4.8 Бургер-меню

Бургер-меню состоит из кнопки и раскрывающейся панели навигации. Кнопка должна сообщать состояние через `aria-expanded` и быть связана с панелью через `aria-controls`.

```
<button
  type="button"
  aria-expanded={open}
  aria-controls="main-menu-panel"
  onClick={() => setOpen(true)}
>
  Меню
</button>
```

Активный маршрут определяется `usePathname`. Официальная документация Next.js описывает `usePathname` как Client Component hook для чтения текущего `pathname`. Внутренние переходы выполняются через `Link`.

Открытая панель использует те же принципы, что и модальный диалог: Escape, блокировка прокрутки, доступное имя и возврат фокуса. Полная адаптивная настройка размеров и контрольных точек будет продолжена в лабораторной работе № 4.

4.9 Изображения в Next.js

Официальная документация Next.js Image Optimization указывает, что компонент `Image` расширяет `img` и обеспечивает выдачу подходящего размера, визуальную стабильность и отложенную загрузку изображений. Локальные статические файлы можно хранить в `public` и указывать от корня URL.

```
import Image from "next/image";

<Image
  src="/images/places/library.jpg"
  alt="Читальный зал научной библиотеки"
  width={900}
  height={600}
/>
```

Свойство `alt` описывает содержание и назначение изображения. Декоративное изображение может иметь `alt=""`, но фотография предметной сущности в каталоге обычно требует осмысленного текста.

`width` и `height` позволяют Next.js определить соотношение сторон и зарезервировать место. Если применяется `fill`, родительский контейнер должен задавать размеры и позиционирование.

В этой работе используются локальные изображения. Для удалённых источников официальная документация требует явно настроить `remotePatterns` в `next.config` и задавать ограничения как можно точнее.

4.10 JavaScript API Яндекс Карт

Официальная документация Яндекс Карт определяет JavaScript API как набор JavaScript-компонентов для публикации интерактивных карт на веб-страницах. API загружается внешним HTTPS-скриптом с параметрами версии, языка и ключа.

```
https://api-maps.yandex.ru/2.1/?apikey=КЛЮЧ&lang=ru_RU
```

Документация Enabling the API указывает, что компоненты загружаются асинхронно. Готовность должна проверяться через `ymaps.ready` или параметр `onload`; одного события готовности DOM недостаточно.

Класс `Map` создаёт карту внутри существующего HTML-элемента. Состояние включает `center` и `zoom`.

```
const map = new ymaps.Map(container, {
  center: [55.75, 37.62],
  zoom: 12,
});
```

Класс `Placemark` представляет точечный геообъект. Конструктор получает координаты, данные и параметры отображения. Метка появляется после добавления в глобальную коллекцию `geoObjects` карты.

```
const placemark = new ymaps.Placemark([55.75, 37.62], {
  hintContent: "Учебное место",
});
```

```
map.geoObjects.add(placemark);
```

В React контейнер карты хранится в `useRef`, создание выполняется в `useEffect`, а `cleanup` вызывает `destroy`. Координаты должны быть числами и соответствовать предметной сущности.

4.11 Переменные окружения клиентской части

Официальная документация Next.js Server and Client Components указывает: переменные с префиксом `NEXT_PUBLIC_` включаются в клиентский JavaScript. Поэтому значение `NEXT_PUBLIC_YANDEX_MAPS_API_KEY` нельзя считать серверным секретом.

```
NEXT_PUBLIC_YANDEX_MAPS_API_KEY=ваш_ключ
```

Файл `.env.local` не включается в отчёт целиком и не отправляется в публичный репозиторий. В отчёте ключ закрывается. Для реального проекта ограничения ключа настраиваются в кабинете поставщика API.

4.12 Условный интерфейс и авторизация

В модальном окне учебный пользователь может нажать «Добавить в избранное», а гость видит ссылку на вход. Это демонстрирует условный рендеринг и общий контекст.

Официальное руководство Next.js Authentication предупреждает, что ограничения клиентского интерфейса недостаточны для безопасности. Серверное действие, `Route Handler` и доступ к данным должны выполнять собственную проверку сессии и прав.

В этой работе избранное хранится в `Set` внутри `useState`. Оно не является пользовательскими данными постоянного хранения. В лабораторных работах № 6 и № 7 действие будет соединено с базой данных и серверной проверкой пользователя.

5. Практическое задание

Этап 1. Подготовка фотографий

Выберите не менее трёх собственных или правомерно используемых фотографий, соответствующих предметной области проекта.

Создайте каталог:

```
public/images/places
```

Для примера StudySpace используются файлы:

```
public/images/places/library.jpg  
public/images/places/coworking.jpg  
public/images/places/campus.jpg
```

Студент должен заменить имена файлов и изображения. В отчёте указывается автор или источник, если это требуется лицензией. Нельзя копировать фотографии из TheraCity.

Этап 2. Расширение типа сущности

Измените `app/data/places.ts`. Добавьте поля `image`, `imageAlt`, `latitude` и `longitude`.

Для каждой сущности заполните:

- путь к фотографии от корня сайта;
- осмысленный `alt`;
- широту;
- долготу.

Координаты должны соответствовать реальному или демонстрационному месту в рамках выбранной темы. Если предметная область не предполагает географических объектов, заранее согласуйте с преподавателем географически осмысленную связанную сущность.

Этап 3. Галерея с `next/image`

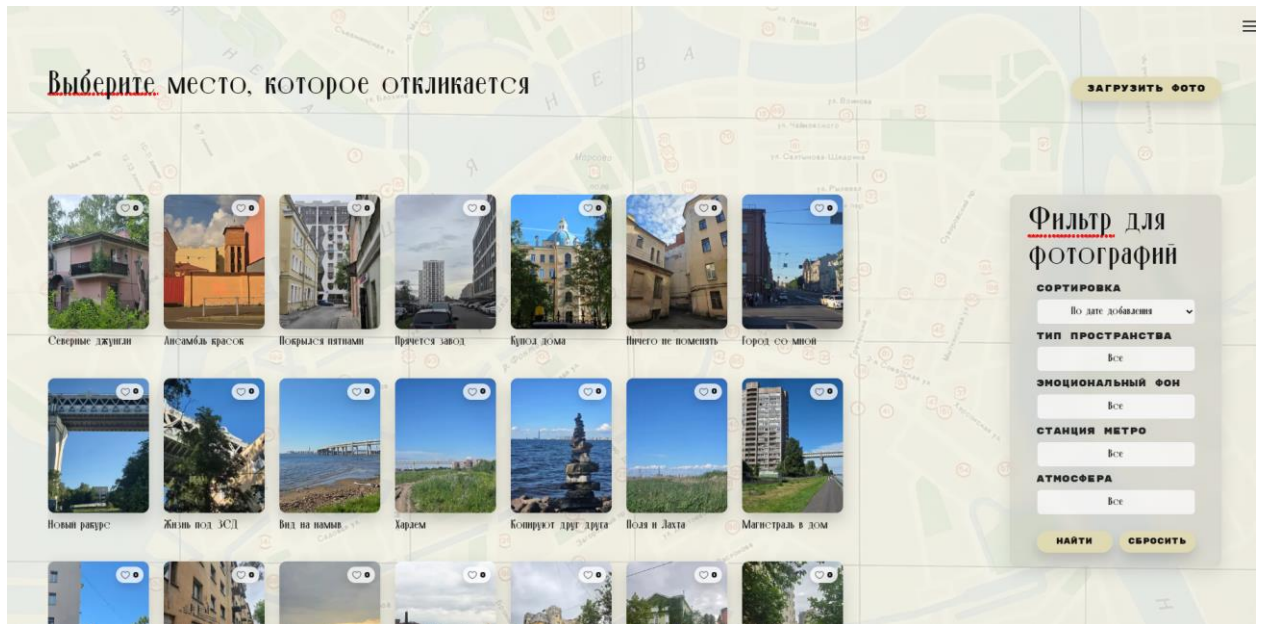
Создайте `app/gallery/page.tsx` и `app/gallery/_components/PlacesGallery.tsx`.

`page.tsx` должен оставаться Server Component. `PlacesGallery` должен быть Client Component, потому что хранит выбранный объект и обработчики карточек.

Каждая карточка должна:

- быть семантической кнопкой;
- содержать `Image`;
- иметь название и краткое описание;
- сохранять ссылку на нажатую кнопку для возврата фокуса;
- открывать модальное окно выбранного объекта.

Добавьте ссылку /gallery в навигацию.



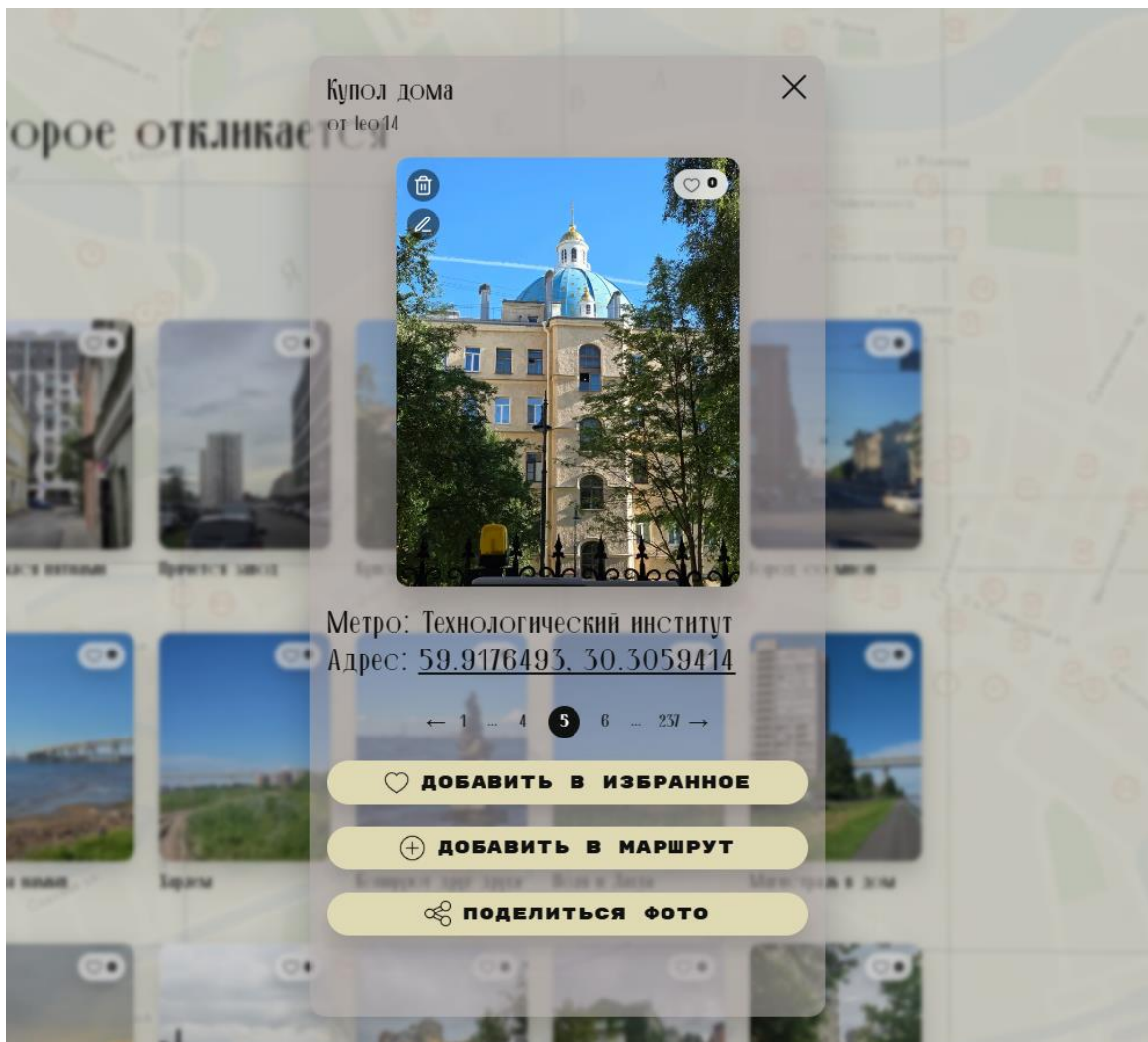
Этап 4. Модальное окно

Создайте `app/gallery/_components/PhotoModal.tsx` и модуль стилей.

Обязательные элементы:

- `createPortal(..., document.body);`
- затемнённый фон;
- `role="dialog"`;
- `aria-modal="true"`;
- `aria-labelledby`, связанный с заголовком;
- видимая кнопка закрытия;
- закрытие `Escape`;
- закрытие нажатием только на фон;
- начальный фокус внутри окна;
- циклический переход `Tab` и `Shift + Tab`;
- возврат фокуса после закрытия;
- блокировка прокрутки документа на время открытия;
- `cleanup` всех обработчиков и изменённых стилей.

Проверьте окно без мыши: открыть карточку клавишей `Enter`, пройти элементы `Tab`, закрыть `Escape` и убедиться, что фокус вернулся на карточку.



Этап 5. Учебное избранное и состояние пользователя

В PlacesGallery создайте Set строковых id в useState.

Если DemoAuthProvider содержит пользователя, модальное окно показывает кнопку добавления или удаления из избранного. Если пользователь отсутствует, показывается Link на /auth/login.

Добавьте текстовое примечание, что избранное хранится только до обновления страницы. Не используйте localStorage как замену серверной БД в этой работе.

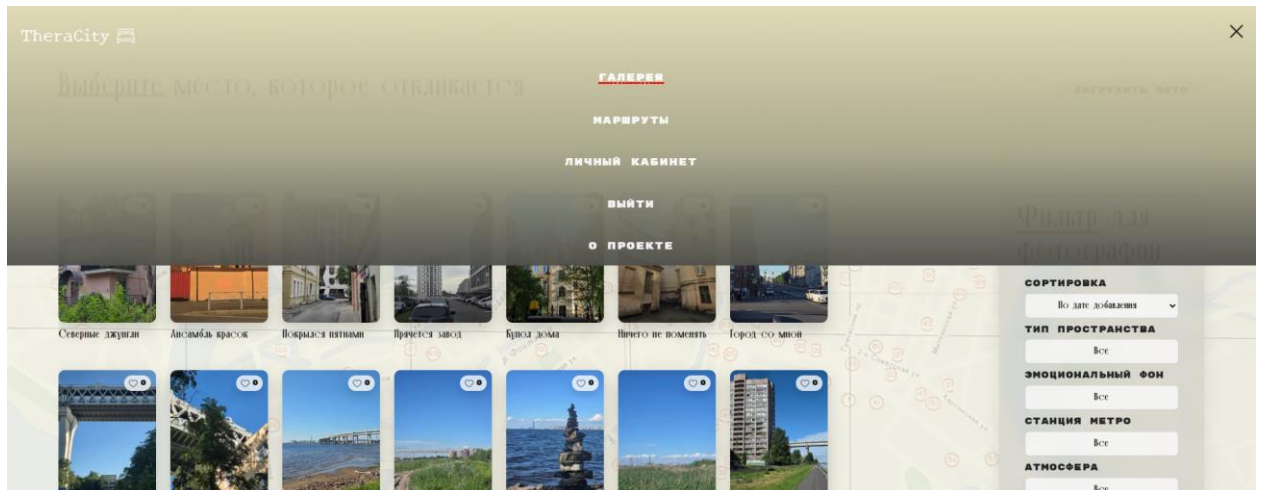
Этап 6. Бургер-меню

Создайте app/ui/BurgerMenu/BurgerMenu.tsx и styles.module.css. Замените AppNav в app/layout.tsx на BurgerMenu.

Меню должно:

- открываться кнопкой;
- иметь aria-expanded и aria-controls;
- содержать Link на start, catalog, gallery, about и cabinet;

- определять активный маршрут через usePathname;
- показывать вход для гостя;
- показывать имя и выход для учебного пользователя;
- закрываться после выбора ссылки;
- закрываться Escape и нажатием на фон;
- блокировать прокрутку при открытии;
- возвращать фокус на кнопку открытия.



Этап 7. Получение и настройка ключа карты

Получите ключ JavaScript API Яндекс Карт в кабинете разработчика, следуя актуальной официальной инструкции.

Создайте файл .env.local в корне проекта:

```
NEXT_PUBLIC_YANDEX_MAPS_API_KEY=ваш_ключ
```

Перезапустите prn run dev после изменения переменной окружения. Не помещайте настоящий ключ в текст отчёта или приложение к отчёту.

Этап 8. Компонент карты

Создайте app/gallery/_components/PhotoMap.tsx и PhotoMap.module.css.

Компонент должен:

- получить массив типизированных объектов;
- использовать ref для контейнера;
- загрузить API по HTTPS;
- дождаться ymaps.ready;
- создать Map;
- создать Placemark для каждого объекта;
- добавить метки в geoObjects;
- передать название в hintContent;

- открыть модальное окно при нажатии метки;
- показать понятное сообщение при отсутствии ключа или ошибке;
- уничтожить карту в cleanup.

Не создавайте новый экземпляр карты при каждом рендере. Эффект зависит от массива объектов и обработчика выбора.

Этап 9. Интеграция карты и галереи

Разместите PhotoMap после сетки карточек внутри PlacesGallery. При нажатии на метку должен устанавливаться тот же selected, что и при нажатии на карточку.

Для открытия с карты returnFocus может быть null, поскольку метка управляется внешним canvas/DOM API. После закрытия фокус должен оставаться в логичном месте страницы.

Проверьте, что карта не перекрывает модальное окно. Значение z-index модального overlay должно быть выше панели карты и меню.

Этап 10. Стилизация клиентской части

Создайте отдельные CSS Modules для галереи, модального окна, карты и бургер-меню.

Требования к стилям:

- карточки имеют одинаковое визуальное соотношение фотографий;
- изображения используют object-fit: cover;
- элементы управления имеют видимый :focus-visible;
- overlay занимает весь viewport;
- модальное окно не выходит за видимую высоту и допускает прокрутку содержимого;
- карта имеет явную высоту;
- закрытое меню не перехватывает события указателя.

Полная система контрольных точек и Lighthouse-проверка выполняются в лабораторной работе № 4. В этой работе достаточно работоспособного базового интерфейса.

Этап 11. Проверка пользовательских сценариев

Запустите:

```
npm run dev
```

Проверьте:

- все фотографии загружаются без 404;
- alt соответствует фотографии;
- каждая карточка открывает правильный объект;
- модальное окно работает мышью и клавиатурой;
- фон страницы не прокручивается при открытом окне;

- фокус не выходит из окна по Tab;
- бургер-меню закрывается всеми предусмотренными способами;
- активный пункт соответствует URL;
- гость видит ссылку на вход;
- учебный пользователь может изменить состояние избранного;
- карта создаётся после загрузки API;
- все метки соответствуют координатам;
- нажатие метки открывает правильное модальное окно;
- после обновления страницы учебный пользователь и избранное сбрасываются.

Остановите сервер и выполните:

```
npm run lint
npm run build
```

6. Требования к результату

После выполнения работы студент должен предоставить проект, в котором:

- сохранена архитектура App Router из лабораторной работы № 2;
- предметная сущность содержит фотографию и координаты;
- в public добавлены не менее трёх изображений;
- фотографии выводятся через next/image;
- alt заполнен осмысленно;
- галерея реализована отдельным Client Component;
- выбор объекта хранится в типизированном состоянии;
- модальное окно создано через createPortal;
- диалог имеет role, aria-modal и доступное имя;
- Escape, Tab, Shift + Tab и возврат фокуса работают корректно;
- все эффекты имеют необходимую очистку;
- бургер-меню использует Link и usePathname;
- кнопка меню имеет aria-expanded и aria-controls;
- интерфейс учитывает учебное состояние пользователя;
- условное избранное не называется защищённой серверной функцией;
- карта создаётся с помощью официального JavaScript API;
- на карте присутствуют метки всех объектов с координатами;
- нажатие метки открывает связанное модальное окно;
- отсутствие ключа обрабатывается понятным сообщением;
- npm run lint и npm run build завершаются без ошибок.

7. Содержание отчёта

Отчёт должен содержать:

- название и цель работы;
- описание реализованного клиентского сценария;
- обновлённую структуру каталога app;
- таблицу свойств предметной сущности;
- сведения об источниках фотографий;
- код массива данных;
- код галереи;
- код модального окна;
- объяснение `useState`, `useEffect`, `useRef` и `createPortal`;
- таблицу требований доступности диалога и их реализации;
- код бургер-меню;
- код компонента карты;
- объяснение асинхронной загрузки `umaps.ready`;
- скриншот галереи;
- скриншот модального окна;
- скриншот открытого бургер-меню;
- скриншот гостевого действия в модальном окне;
- скриншот действия после учебного входа;
- скриншот карты с метками;
- скриншот сообщения об отсутствии ключа или описание его обработки;
- результаты `npm run lint` и `npm run build`;
- объяснение, почему `NEXT_PUBLIC_` значение видно клиенту;
- объяснение, почему условная кнопка не является авторизацией;
- ответы на контрольные вопросы;
- вывод по работе.

Контрольные вопросы:

- Для чего требуется директива `"use client"`?
- Почему `page.tsx` галереи может оставаться Server Component?
- Что возвращает `useState`?
- Для каких задач используется `useRef`?
- Когда выполняется `cleanup` `useEffect`?
- Чем портал изменяет физическое размещение DOM?
- Почему контекст остаётся доступен внутри портала?
- Какие атрибуты нужны контейнеру модального диалога?
- Как должны работать `Tab`, `Shift + Tab` и `Escape`?
- Почему для карточки используется `button`, а не `div` с `onClick`?
- Какие задачи решает `next/image`?
- Где хранятся локальные изображения?
- Почему API карты загружается асинхронно?
- Для чего используется `umaps.ready`?
- Как `Placemark` добавляется на карту?

- Почему NEXT_PUBLIC_YANDEX_MAPS_API_KEY не является серверным секретом?
- Почему скрытие действия для гостя не защищает данные?

8. Источники

Теоретическая часть подготовлена по официальной документации. Архитектурные примеры адаптированы к последовательности курса и реализации клиентских компонентов TheraCity.

- Next.js: Server and Client Components — <https://nextjs.org/docs/app/getting-started/server-and-client-components>
- Next.js: use client — <https://nextjs.org/docs/app/api-reference/directives/use-client>
- Next.js: usePathname — <https://nextjs.org/docs/app/api-reference/functions/use-pathname>
- Next.js: Linking and Navigating — <https://nextjs.org/docs/app/getting-started/linking-and-navigating>
- Next.js: Image Optimization — <https://nextjs.org/docs/app/getting-started/images>
- Next.js: Image Component — <https://nextjs.org/docs/app/api-reference/components/image>
- Next.js: Authentication — <https://nextjs.org/docs/app/guides/authentication>
- React: useState — <https://react.dev/reference/react/useState>
- React: useEffect — <https://react.dev/reference/react/useEffect>
- React: useRef — <https://react.dev/reference/react/useRef>
- React: Responding to Events — <https://react.dev/learn/responding-to-events>
- React: Conditional Rendering — <https://react.dev/learn/conditional-rendering>
- React DOM: createPortal — <https://react.dev/reference/react-dom/createPortal>
- W3C WAI-ARIA APG: Dialog (Modal) Pattern — <https://www.w3.org/WAI/ARIA/apg/patterns/dialog-modal/>
- Yandex Maps JavaScript API: General Information — <https://yandex.com/dev/jsapi-v2-1/doc/en/v2-1/dg/concepts/general>
- Yandex Maps JavaScript API: Enabling the API — <https://yandex.com/dev/jsapi-v2-1/doc/en/v2-1/dg/concepts/load>
- Yandex Maps JavaScript API: Map — <https://yandex.com/dev/jsapi-v2-1/doc/en/v2-1/ref/reference/Map>
- Yandex Maps JavaScript API: Placemark — <https://yandex.com/dev/jsapi-v2-1/doc/en/v2-1/ref/reference/Placemark>

1. Обновлённая структура файлов

Показывает новые компоненты галереи, модального окна, меню и карты.

```
app/
├── gallery/
│   ├── components/
│   │   ├── PhotoMap.module.css
│   │   ├── PhotoMap.tsx
│   │   ├── PhotoModal.module.css
│   │   ├── PhotoModal.tsx
│   │   ├── PlacesGallery.module.css
│   │   └── PlacesGallery.tsx
│   └── page.tsx
├── ui/
│   └── BurgerMenu/
│       ├── BurgerMenu.tsx
│       └── styles.module.css
├── data/
│   └── places.ts
├── providers/
│   └── DemoAuthProvider.tsx
├── layout.tsx
└── ... файлы лабораторной работы № 2
```

```
public/
├── images/
│   └── places/
│       ├── campus.jpg
│       ├── coworking.jpg
│       └── library.jpg
```

.env.local

2. Обновлённый файл app/data/places.ts (этап 2)

// Расширяет учебное место фотографией, альтернативным текстом и координатами.

// Описываем поля одной предметной сущности.

```
export type StudyPlace = {
  id: string;
  title: string;
  summary: string;
  address: string;
  image: string;
  imageAlt: string;
  latitude: number;
  longitude: number;
};
```

// Храним демонстрационный набор объектов в типизированном массиве.

```
export const studyPlaces: StudyPlace[] = [
  {
    id: "library",
    title: "Научная библиотека",
    summary: "Тихое место для индивидуальной работы с книгами и конспектами.",
    address: "Университетская улица, 1",
    image: "/images/places/library.jpg",
```

```

        imageAlt: "Читальный зал научной библиотеки",
        latitude: 55.7544,
        longitude: 37.6192,
      },
      {
        id: "coworking",
        title: "Студенческий коворкинг",
        summary: "Пространство для групповых проектов и обсуждений.",
        address: "Учебный корпус, аудитория 210",
        image: "/images/places/coworking.jpg",
        imageAlt: "Столы для групповой работы в студенческом коворкинге",
        latitude: 55.7583,
        longitude: 37.6106,
      },
      {
        id: "campus",
        title: "Открытая площадка кампуса",
        summary: "Место для чтения и коротких учебных встреч на свежем воздухе.",
        address: "Центральный двор кампуса",
        image: "/images/places/campus.jpg",
        imageAlt: "Открытая площадка с местами для работы в кампусе",
        latitude: 55.7498,
        longitude: 37.6302,
      },
    ],
  };

  export function getStudyPlace(id: string): StudyPlace | undefined {
    return studyPlaces.find((place) => place.id === id);
  }

```

3. Файл app/gallery/page.tsx (этап 3)

```

// Подключает интерактивную галерею к серверной странице маршрута.

import PlacesGallery from "../_components/PlacesGallery";

// Объявляем основной компонент этого файла.
export default function GalleryPage() {
  // Возвращаем JSX-разметку, которую React выведет на странице.
  return (
    <main>
      <h1>Галерея учебных мест</h1>
      <p>
        Выберите карточку или метку на карте, чтобы открыть подробное окно.
      </p>
      <PlacesGallery />
    </main>
  );
}

```

4. Файл app/gallery/_components/PlacesGallery.tsx (этапы 3, 5 и 9)

```

// Управляет карточками, выбранным объектом, избранным, модальным окном и картой.

"use client";

import Image from "next/image";
import { useRef, useState } from "react";
import { studyPlaces, type StudyPlace } from "../../data/places";

```

```

import PhotoMap from "./PhotoMap";
import PhotoModal from "./PhotoModal";
import styles from "./PlacesGallery.module.css";

// Объявляем основной компонент этого файла.
export default function PlacesGallery() {
  // Создаём локальное состояние компонента.
  const [selected, setSelected] = useState<StudyPlace | null>(null);
  const [favoriteIds, setFavoriteIds] = useState<Set<string>>(
    () => new Set<string>(),
  );
  const triggerRef = useRef<HTMLButtonElement | null>(null);
  const [returnFocusTo, setReturnFocusTo] = useState<HTMLButtonElement | null>(null);

  function openFromCard(place: StudyPlace, button: HTMLButtonElement) {
    triggerRef.current = button;
    setReturnFocusTo(button);
    setSelected(place);
  }

  function openFromMap(place: StudyPlace) {
    triggerRef.current = null;
    setReturnFocusTo(null);
    setSelected(place);
  }

  function closeModal() {
    setSelected(null);
  }

  function toggleFavorite(id: string) {
    setFavoriteIds((current) => {
      const next = new Set(current);
      if (next.has(id)) next.delete(id);
      else next.add(id);
      return next;
    });
  }

  // Возвращаем JSX-разметку, которую React выведет на странице.
  return (
    <>
      <div className={styles.grid}>
        {studyPlaces.map((place) => (
          <button
            className={styles.card}
            type="button"
            key={place.id}
            onClick={(event) => openFromCard(place, event.currentTarget)}
          >
            <span className={styles.imageWrap}>
              <Image
                src={place.image}
                alt={place.imageAlt}
                fill
                sizes="(max-width: 700px) 100vw, 33vw"
                className={styles.image}
              />
            </span>
            <span className={styles.title}>{place.title}</span>
            <span className={styles.summary}>{place.summary}</span>
          </button>
        ))}
      </div>

      <p className={styles.demoNote}>
        Избранное в этой работе хранится только до обновления страницы.
      </p>
    </>
  );
}

```

```

        <PhotoMap places={studyPlaces} onSelect={openFromMap} />

        {selected ? (
          <PhotoModal
            place={selected}
            isFavorite={favoriteIds.has(selected.id)}
            onToggleFavorite={() => toggleFavorite(selected.id)}
            onClose={closeModal}
            returnFocusTo={returnFocusTo}
          />
        ) : null}
      </>
    );
  }
}

```

5. Файл app/gallery/_components/PlacesGallery.module.css

```

/* Оформляет адаптивную сетку карточек и состояния галереи. */

/* Создаём адаптивную сетку карточек. */
.grid {
  display: grid;
  grid-template-columns: repeat(auto-fit, minmax(240px, 1fr));
  gap: 18px;
  margin-top: 24px;
}

/* Оформляем интерактивную карточку объекта. */
.card {
  display: grid;
  gap: 10px;
  overflow: hidden;
  padding: 0 0 18px;
  border: 1px solid #d7d2c8;
  border-radius: 14px;
  background: #ffffff;
  color: inherit;
  text-align: left;
}

/* Показываем заметный фокус при клавиатурной навигации. */
.card:focus-visible {
  outline: 4px solid #1d70a2;
  outline-offset: 4px;
}

/* Резервируем область изображения с постоянными пропорциями. */
.imageWrap {
  position: relative;
  display: block;
  aspect-ratio: 3 / 2;
  overflow: hidden;
  background: #d9dde2;
}

.image {
  object-fit: cover;
}

.title,
.summary {

```

```

padding: 0 16px;
}

.title {
font-size: 20px;
font-weight: 800;
}

.summary {
line-height: 1.5;
}

.demoNote {
margin: 18px 0 32px;
color: #5b6470;
}

```

6. Файл app/gallery/_components/PhotoModal.tsx (этапы 4–5)

```

// Создаёт доступное модальное окно с изображением, действиями и управлением фокусом.

"use client";

import Image from "next/image";
import Link from "next/link";
import { useEffect, useRef, type KeyboardEvent as ReactKeyboardEvent } from "react";
import { createPortal } from "react-dom";
import { useDemoAuth } from "../../providers/DemoAuthProvider";
import type { StudyPlace } from "../../data/places";
import styles from "./PhotoModal.module.css";

// Типизируем входные свойства компонента.
type PhotoModalProps = {
  place: StudyPlace;
  isFavorite: boolean;
  onToggleFavorite: () => void;
  onClose: () => void;
  returnFocusTo: HTMLElement | null;
};

// Собираем селектор элементов, которые могут получать фокус.
const focusableSelector = [
  "a[href]",
  "button:not([disabled])",
  "input:not([disabled])",
  "select:not([disabled])",
  "textarea:not([disabled])",
  "[tabindex]:not([tabindex='-1'])",
].join(",");

// Объявляем основной компонент этого файла.
export default function PhotoModal({
  place,
  isFavorite,
  onToggleFavorite,
  onClose,
  returnFocusTo,
}: PhotoModalProps) {
  const { user } = useDemoAuth();
  const dialogRef = useRef<HTMLDivElement>(null);
  const closeButtonRef = useRef<HTMLButtonElement>(null);

```

```

// Синхронизируем компонент с браузерными событиями или внешним API.
useEffect(() => {
  const previousOverflow = document.documentElement.style.overflow;
  document.documentElement.style.overflow = "hidden";
  closeButtonRef.current?.focus();

  function handleEscape(event: globalThis.KeyboardEvent) {
    if (event.key === "Escape") onClose();
  }

  window.addEventListener("keydown", handleEscape);

  // В cleanup освобождаем подписки и внешние ресурсы эффекта.
  return () => {
    window.removeEventListener("keydown", handleEscape);
    document.documentElement.style.overflow = previousOverflow;
    returnFocusTo?.focus();
  };
}, [onClose, returnFocusTo]);

// Не выпускаем клавиатурный фокус за границы открытого интерфейса.
function keepFocusInside(event: ReactKeyboardEvent<HTMLDivElement>) {
  if (event.key !== "Tab") return;

  const focusable = Array.from(
    dialogRef.current?.querySelectorAll<HTMLElement>(focusableSelector) ?? [],
  );

  if (focusable.length === 0) {
    // Отменяем стандартное действие события и выполняем собственную логику.
    event.preventDefault();
    return;
  }

  const first = focusable[0];
  const last = focusable[focusable.length - 1];

  if (event.shiftKey && document.activeElement === first) {
    event.preventDefault();
    last.focus();
  } else if (!event.shiftKey && document.activeElement === last) {
    event.preventDefault();
    first.focus();
  }
}

// Рендерим окно в document.body поверх основного интерфейса.
return createPortal(
  <div
    className={styles.overlay}
    onMouseDown={(event) => {
      if (event.target === event.currentTarget) onClose();
    }}
  >
    <div
      ref={dialogRef}
      className={styles.dialog}
      role="dialog"
      aria-modal="true"
      aria-labelledby="place-modal-title"
      onKeyDown={keepFocusInside}
    >
      <button
        ref={closeButtonRef}
        className={styles.close}
        type="button"
        aria-label="Заккрыть окно"
        onClick={onClose}

```

```

    >
      x
    </button>

    <h2 id="place-modal-title">{place.title}</h2>

    <Image
      src={place.image}
      alt={place.imageAlt}
      width={900}
      height={600}
      className={styles.image}
      sizes="(max-width: 700px) calc(100vw - 48px), 720px"
    />

    <p>{place.summary}</p>
    <p>
      <strong>Адрес:</strong> {place.address}
    </p>

    {user ? (
      <button type="button" onClick={onToggleFavorite}>
        {isFavorite ? "Удалить из избранного" : "Добавить в избранное"}
      </button>
    ) : (
      <Link href="/auth/login">Войдите, чтобы использовать избранное</Link>
    )}
  </div>
</div>,
document.body,
);
}

```

7. Файл app/gallery/_components/PhotoModal.module.css

```

/* Размещает модальное окно поверх страницы и ограничивает его видимой областью. */

/* Перекрываем viewport затемнённым фоном. */
.overlay {
  position: fixed;
  inset: 0;
  z-index: 3000;
  display: grid;
  place-items: center;
  padding: 24px;
  background: rgb(15 23 42 / 72%);
}

/* Ограничиваем размеры окна и разрешаем прокрутку длинного содержимого. */
.dialog {
  position: relative;
  width: min(760px, 100%);
  max-height: calc(100dvh - 48px);
  overflow-y: auto;
  padding: 28px;
  border-radius: 16px;
  background: #ffffff;
  box-shadow: 0 24px 70px rgb(0 0 0 / 35%);
}

.dialog h2 {

```

```

    margin: 0 48px 18px 0;
}

.close {
  position: absolute;
  top: 14px;
  right: 14px;
  width: 40px;
  height: 40px;
  border: 0;
  border-radius: 50%;
  background: #e8edf2;
  font-size: 28px;
  line-height: 1;
}

/* Показываем заметный фокус при клавиатурной навигации. */
.close:focus-visible,
.dialog a:focus-visible,
.dialog button:focus-visible {
  outline: 4px solid #1d70a2;
  outline-offset: 3px;
}

.image {
  width: 100%;
  height: auto;
  border-radius: 12px;
  object-fit: cover;
}

```

8. Файл app/ui/BurgerMenu/BurgerMenu.tsx (этап 6)

```

// Реализует доступное бургер-меню с маршрутами и состоянием пользователя.

"use client";

import Link from "next/link";
import { usePathname } from "next/navigation";
import {
  useEffect,
  useRef,
  useState,
  type KeyboardEvent as ReactKeyboardEvent,
} from "react";
import { useDemoAuth } from "../../providers/DemoAuthProvider";
import styles from "../../styles/module.css";

// Описываем пункты меню единым массивом.
const links = [
  { href: "/start", label: "Главная" },
  { href: "/catalog", label: "Каталог" },
  { href: "/gallery", label: "Галерея и карта" },
  { href: "/about", label: "О проекте" },
  { href: "/cabinet", label: "Кабинет" },
];

// Собираем селектор доступных элементов внутри меню.
const menuFocusableSelector = [
  "a[href]",
  "button:not([disabled])",

```

```

    '[tabindex]:not([tabindex="-1"])',
  ].join(",");

function isActive(pathname: string, href: string): boolean {
  return pathname === href || pathname.startsWith(`${href}/`);
}

// Объявляем основной компонент этого файла.
export default function BurgerMenu() {
  const pathname = usePathname();
  const { user, logout } = useDemoAuth();
  // Создаём локальное состояние компонента.
  const [open, setOpen] = useState(false);
  const openButtonRef = useRef<HTMLButtonElement>(null);
  const closeButtonRef = useRef<HTMLButtonElement>(null);
  const panelRef = useRef<HTMLElement>(null);

  // Синхронизируем компонент с браузерными событиями или внешним API.
  useEffect(() => {
    if (!open) return;

    const previousOverflow = document.documentElement.style.overflow;
    const openButton = openButtonRef.current;
    document.documentElement.style.overflow = "hidden";
    closeButtonRef.current?.focus();

    function handleEscape(event: KeyboardEvent) {
      if (event.key === "Escape") setOpen(false);
    }

    window.addEventListener("keydown", handleEscape);

    // В cleanup освобождаем подписки и внешние ресурсы эффекта.
    return () => {
      window.removeEventListener("keydown", handleEscape);
      document.documentElement.style.overflow = previousOverflow;
      openButton?.focus();
    };
  }, [open]);

  function closeMenu() {
    setOpen(false);
  }

  // Не выпускаем клавиатурный фокус за границы открытого интерфейса.
  function keepFocusInside(event: ReactKeyboardEvent<HTMLElement>) {
    if (event.key !== "Tab") return;

    const focusable = Array.from(
      panelRef.current?.querySelectorAll<HTMLElement>(menuFocusableSelector) ?? [],
    );
    if (focusable.length === 0) {
      // Отменяем стандартное действие события и выполняем собственную логику.
      event.preventDefault();
      return;
    }

    const first = focusable[0];
    const last = focusable[focusable.length - 1];
    if (event.shiftKey && document.activeElement === first) {
      event.preventDefault();
      last.focus();
    } else if (!event.shiftKey && document.activeElement === last) {
      event.preventDefault();
      first.focus();
    }
  }

  // Возвращаем JSX-разметку, которую React выведет на странице.

```

```

return (
  <>
    <button
      ref={openButtonRef}
      className={styles.openButton}
      type="button"
      aria-label="Открыть меню"
      aria-expanded={open}
      aria-controls="main-menu-panel"
      onClick={() => setOpen(true)}
    >
      <span aria-hidden="true">☰</span>
    </button>

    {open ? (
      <div
        className={styles.overlay}
        onMouseDown={(event) => {
          if (event.target === event.currentTarget) closeMenu();
        }}
      >
        <section
          ref={panelRef}
          id="main-menu-panel"
          className={styles.panel}
          role="dialog"
          aria-modal="true"
          aria-label="Основное меню"
          onKeyDown={keepFocusInside}
        >
          <div className={styles.header}>
            <Link href="/start" onClick={closeMenu}>
              StudySpace
            </Link>
            <button
              ref={closeButtonRef}
              type="button"
              aria-label="Закрыть меню"
              onClick={closeMenu}
            >
              x
            </button>
          </div>

          <nav aria-label="Разделы приложения">
            <ul className={styles.list}>
              {links.map((link) => (
                <li key={link.href}>
                  <Link
                    href={link.href}
                    aria-current={isActive(pathname, link.href) ? "page" : undefined}
                    onClick={closeMenu}
                  >
                    {link.label}
                  </Link>
                </li>
              ))}
            </ul>
          </nav>

          <div className={styles.authRow}>
            {user ? (
              <>
                <span>Пользователь: {user.name}</span>
                <button
                  type="button"
                  onClick={() => {
                    logout();
                    closeMenu();

```

```

        }}
      >
        Выйти
      </button>
    </>
  ) : (
    <Link href="/auth/login" onClick={closeMenu}>
      Войти / зарегистрироваться
    </Link>
  )}
</div>
</section>
</div>
) : null}
</>
);
}

```

9. Файл

app/ui/BurgerMenu/styles.module.css

```

/* Оформляет кнопку меню, затемнение и выезжающую панель навигации. */

/* Закрепляем кнопку открытия меню поверх страницы. */
.openButton {
  position: fixed;
  top: 16px;
  right: 16px;
  z-index: 2000;
  width: 48px;
  height: 48px;
  border: 1px solid #b9c0c8;
  border-radius: 50%;
  background: #ffffff;
  font-size: 24px;
}

/* Показываем заметный фокус при клавиатурной навигации. */
.openButton:focus-visible,
.panel a:focus-visible,
.panel button:focus-visible {
  outline: 4px solid #1d70a2;
  outline-offset: 3px;
}

/* Перекрываем viewport затемнённым фоном. */
.overlay {
  position: fixed;
  inset: 0;
  z-index: 2200;
  background: rgb(15 23 42 / 55%);
}

/* Размещаем панель меню у правого края viewport. */
.panel {
  width: min(430px, 100%);
  min-height: 100%;
  margin-left: auto;
  padding: 24px;
  background: #ffffff;
  box-shadow: -20px 0 50px rgb(0 0 0 / 25%);
}

.header,
.authRow {

```

```

    display: flex;
    align-items: center;
    justify-content: space-between;
    gap: 16px;
  }

  .header a {
    color: #1f2933;
    font-size: 24px;
    font-weight: 800;
    text-decoration: none;
  }

  .header button {
    width: 42px;
    height: 42px;
    border: 0;
    border-radius: 50%;
    font-size: 28px;
  }

  .list {
    display: grid;
    gap: 10px;
    margin: 48px 0;
    padding: 0;
    list-style: none;
  }

  .list a {
    display: block;
    padding: 12px;
    border-radius: 10px;
    color: #1f2933;
    font-size: 20px;
    text-decoration: none;
  }

  .list a[aria-current="page"] {
    background: #dbeafe;
    color: #123b5d;
    font-weight: 800;
  }

  .authRow {
    padding-top: 20px;
    border-top: 1px solid #d7d2c8;
  }

```

10. Обновлённый файл app/layout.tsx (этап 6)

```

// Подключает бургер-меню и учебный провайдер к корневому layout.

import type { Metadata } from "next";
import type { ReactNode } from "react";
import DemoAuthProvider from "../providers/DemoAuthProvider";
import BurgerMenu from "../ui/BurgerMenu/BurgerMenu";
import "../globals.css";

// Задаём заголовок и описание страницы для Metadata API.
export const metadata: Metadata = {
  title: "StudySpace",
  description: "Учебный проект о местах для самостоятельной работы",
};

```

```

// Объявляем основной компонент этого файла.
export default function RootLayout({
  children,
}: Readonly<{ children: ReactNode }>) {
  // Возвращаем JSX-разметку, которую React выведет на странице.
  return (
    <html lang="ru">
      <body>
        <DemoAuthProvider>
          <BurgerMenu />
          {children}
        </DemoAuthProvider>
      </body>
    </html>
  );
}

```

11. Файл app/gallery/_components/PhotoMap.tsx (этап 8)

```

// Загружает API Яндекс Карт, создаёт карту и связывает метки с объектами.

"use client";

import { useEffect, useRef, useState } from "react";
import type { StudyPlace } from "../../data/places";
import styles from "../PhotoMap.module.css";

// Координаты представлены парой широты и долготы.
type Coordinates = [number, number];

type YandexPlacemark = {
  events: {
    add: (eventName: string, handler: () => void) => void;
  };
};

type YandexMap = {
  geoObjects: {
    add: (placemark: YandexPlacemark) => void;
  };
  destroy: () => void;
};

type YandexMapsApi = {
  ready: (callback: () => void) => void;
  Map: new (
    container: HTMLElement,
    state: { center: Coordinates; zoom: number; controls: string[] },
  ) => YandexMap;
  Placemark: new (
    coordinates: Coordinates,
    properties: { hintContent: string; balloonContent: string },
    options: { preset: string; iconColor: string },
  ) => YandexPlacemark;
};

// Дополняем тип Window объектом, который добавляет внешний API.
declare global {
  interface Window {
    ymaps?: YandexMapsApi;
  }
}

```

```

    }
  }

  const scriptId = "study-space-yandex-maps";
  let mapsPromise: Promise<YandexMapsApi> | null = null;

  // Загружаем скрипт карты один раз и ожидаем готовности API.
  function loadMaps(apiKey: string): Promise<YandexMapsApi> {
    if (window.ymaps) {
      return new Promise((resolve) => {
        window.ymaps?.ready(() => resolve(window.ymaps as YandexMapsApi));
      });
    }

    if (mapsPromise) return mapsPromise;

    mapsPromise = new Promise((resolve, reject) => {
      function resolveWhenReady() {
        if (!window.ymaps) {
          mapsPromise = null;
          reject(new Error("Yandex Maps API is unavailable"));
          return;
        }

        window.ymaps.ready(() => resolve(window.ymaps as YandexMapsApi));
      }

      const existing = document.getElementById(scriptId) as HTMLScriptElement | null;
      if (existing) {
        existing.addEventListener("load", resolveWhenReady, { once: true });
        existing.addEventListener("error", () => reject(new Error("Map script error")), {
          once: true,
        });
        return;
      }

      const script = document.createElement("script");
      script.id = scriptId;
      script.src = `https://api-
maps.yandex.ru/2.1/?apikey=${encodeURIComponent(apiKey)}&lang=ru_RU`;
      script.async = true;
      script.addEventListener("load", resolveWhenReady, { once: true });
      script.addEventListener(
        "error",
        () => {
          mapsPromise = null;
          reject(new Error("Map script error"));
        },
        { once: true },
      );
      // Подключаем внешний скрипт к документу.
      document.head.appendChild(script);
    });

    return mapsPromise;
  }

  // Объявляем основной компонент этого файла.
  export default function PhotoMap({
    places,
    onSelect,
  }: {
    places: StudyPlace[];
    onSelect: (place: StudyPlace) => void;
  }) {
    const containerRef = useRef<HTMLDivElement>(null);
    // Создаём локальное состояние компонента.
    const [status, setStatus] = useState<"loading" | "ready" | "error" | "no-key">(
      "loading",
    );

```

```

);
const apiKey = process.env.NEXT_PUBLIC_YANDEX_MAPS_API_KEY ?? "";

// Синхронизируем компонент с браузерными событиями или внешним API.
useEffect(() => {
  const status = !apiKey ? "no-key" : "loading";

  if (!apiKey) {
    return;
  }

  if (!containerRef.current || places.length === 0) return;

  let disposed = false;
  let map: YandexMap | null = null;

  async function buildMap() {
    try {
      const ymaps = await loadMaps(apiKey);
      if (disposed || !containerRef.current) return;

      const first = places[0];
      // Создаём экземпляр карты в подготовленном DOM-контейнере.
      map = new ymaps.Map(containerRef.current, {
        center: [first.latitude, first.longitude],
        zoom: 12,
        controls: ["zoomControl"],
      });

      // Добавляем на карту метку для каждого учебного места.
      places.forEach((place) => {
        // Создаём метку с координатами и данными текущего объекта.
        const placemark = new ymaps.Placemark(
          [place.latitude, place.longitude],
          {
            hintContent: place.title,
            balloonContent: place.address,
          },
          {
            preset: "islands#circleDotIcon",
            iconColor: "#184e77",
          },
        );

        placemark.events.add("click", () => onSelect(place));
        map?.geoObjects.add(placemark);
      });

      setStatus("ready");
    } catch {
      if (!disposed) setStatus("error");
    }
  }

  void buildMap();

  // В cleanup освобождаем подписки и внешние ресурсы эффекта.
  return () => {
    disposed = true;
    // Уничтожаем экземпляр карты и его обработчики.
    map?.destroy();
  };
}, [apiKey, onSelect, places]);

// Возвращаем JSX-разметку, которую React выведет на странице.
return (
  <section className={styles.section} aria-labelledby="map-title">
    <h2 id="map-title">Карта учебных мест</h2>
  </section>
);

```

```

    {status === "no-key" ? (
      <p className={styles.status}>
        Добавьте NEXT_PUBLIC_YANDEX_MAPS_API_KEY в файл .env.local.
      </p>
    ) : null}

    {status === "error" ? (
      <p className={styles.status}>Не удалось загрузить карту.</p>
    ) : null}

    {status === "loading" ? (
      <p className={styles.status}>Карта загружается...</p>
    ) : null}

    <div
      ref={containerRef}
      className={styles.canvas}
      aria-label="Интерактивная карта учебных мест"
    />
  </section>
);
}

```

12. Файл app/gallery/_components/PhotoMap.module. css

```

/* Задаёт размеры контейнера карты и оформление сообщений состояния. */
.section {
  margin-top: 48px;
}

/* Задаём явные размеры контейнера интерактивной карты. */
.canvas {
  width: 100%;
  height: 480px;
  overflow: hidden;
  border: 1px solid #b9c0c8;
  border-radius: 16px;
  background: #dbe3ea;
}

/* Выделяем сообщения о загрузке или ошибке карты. */
.status {
  padding: 12px 14px;
  border-left: 4px solid #184e77;
  background: #e9f2f8;
}

```

13. Файл .env.local (этап 7)

```

# Хранит публичный учебный ключ JavaScript API карты.

NEXT_PUBLIC_YANDEX_MAPS_API_KEY=вставьте_свой_ключ

```

Настоящее значение не включается в отчёт. После создания или изменения .env.local перезапустите сервер разработки.

14. Проверка кода

```
# Запускает ручную проверку, линтер и production-сборку.  
  
npm run dev  
# после ручной проверки остановить сервер: Ctrl + C  
# После ручной проверки выполняем автоматические проверки проекта.  
npm run lint  
npm run build
```

Если ESLint сообщает о нестабильной зависимости onSelect, передайте в PhotoMap функцию, созданную через useCallback, либо сохраните простую функцию верхнего уровня клиентского компонента. Нельзя отключать правило без объяснения причины.

15. Что студент должен заменить

- тип и содержание предметной сущности;
- фотографии и тексты alt;
- адреса и координаты;
- названия разделов бургер-меню;
- подписи модального окна;
- действие «избранное» на подходящее учебное действие своей темы;
- центр и масштаб карты;
- цветовое оформление CSS Modules.

Должны сохраниться Client Component границы, семантические кнопки, доступный диалог, cleanup эффектов, next/image, официальный картографический API и явное предупреждение об учебном состоянии пользователя.